PROJECT DOCUMENTATION

Spark Event Log Analyzer

A complete, beginner-friendly walkthrough and a system design analysis under load

This guide has two halves. The first assumes you have never opened a Spark event log and walks the entire codebase end to end - what every module does, what flows between them, and a fully worked example traced from eleven lines of JSON to six findings. The second asks what happens to each of those components as load increases: bigger logs, more logs, more users, more rules, and a demand for history. Every performance number in Part 6 was measured by running this repository's code against generated event logs of up to 1.77 GB.

What you get	
Parts 1 to 5	Spark fundamentals, the pipeline, all seven modules, a worked example, and the six detection rules decoded.
Part 6	System design under load: five load axes, measured bottlenecks, a four-level scaling ladder, capacity planning, backpressure, idempotency, security, storage, observability and alert fatigue.
Part 7	Correctness gaps reproduced against the code, including two that are invisible to the current test suite but affect every real Spark log.
Part 8	How to run it, how to extend it, a prioritised work order, and a glossary.

Repository	ajaynalla/spark-event-log-analyzer
Analysed at	branch claude/spark-analyzer-pdf-docs-ehii13
Codebase	520 lines of Python across 7 modules; zero runtime dependencies
Test suite	3 tests, passing
Generated	08 September 2026

Contents

Part 1 - What this project actually is

1.1 The problem, in plain English
1.2 What a Spark event log is
1.3 The four words you need: Application, Job, Stage, Task
1.4 The five failure patterns the tool looks for
1.5 Project facts at a glance

Part 2 - The pipeline, end to end

2.1 The data contract between layers
2.2 What actually executes, in order

Part 3 - Module by module

3.1 models.py - the vocabulary
3.2 parser.py - reading and normalising the log
3.3 analyzer.py - measurement and orchestration
3.4 heuristics.py - the rules engine
3.5 reporting.py - serialisation
3.6 cli.py - the user interface
3.7 __init__.py - the public surface

Part 4 - Worked example: tracing the sample log

4.1 What the 11 lines contain
4.2 Stage 10's four task attempts, as parsed
4.3 The roll-up arithmetic, step by step
4.4 Which rules fire, and why

Part 5 - The six rules, decoded

5.1 What each finding means, and what to actually do

Part 6 - System design: what happens as load increases

6.1 Naming what we have today
6.2 The five independent load dimensions
6.3 Measured scaling behaviour of the current code
6.4 Bottleneck 1: the log is fully materialised (memory)
6.5 Bottleneck 2: five passes over the same data (CPU)
6.6 Bottleneck 3: every task of a stage is held to compute statistics
6.7 Bottleneck 4: no concurrency, and nowhere to put it
6.8 The scaling ladder
6.9 Target architecture at L2 and L3
6.10 Capacity planning: sizing the worker pool
6.11 Backpressure and the heavy tail
6.12 Idempotency and caching - and why they are free here
6.13 Failure handling: all-or-nothing parsing
6.14 Multi-tenancy and security
6.15 Storage, retention and the economics of extraction
6.16 Observability and service levels
6.17 History: the feature that changes what the tool can say
6.18 The real capacity limit at scale is human attention
6.19 What breaks first, in order

Part 7 - Correctness gaps found while writing this document

7.1 HIGH - task status is misread on real Spark logs
7.2 HIGH - retry detection does not work on real Spark logs
7.3 Everything else, ranked

Part 8 - Running it, extending it, and a glossary

8.1 Getting it running
8.2 Adding a seventh rule
8.3 A suggested order of work
8.4 Glossary

Part 1 - What this project actually is

Start here if you have never opened a Spark event log.

1.1 The problem, in plain English

A Spark job that used to finish in 20 minutes now takes 3 hours. Nothing in the code changed. The usual next step is for an engineer to open the Spark History Server UI, click through jobs into stages into task lists, sort task tables by duration, eyeball the shuffle columns, and try to remember what "normal" looked like last week. That investigation is slow, manual, unrepeatable, and it produces a conclusion in someone's head rather than a document.

This project automates exactly that investigation. Spark already writes down everything that happened during a run, in a file called the **event log**. The tool reads that file, computes the same summary statistics a human would compute by squinting at the UI, applies six rules that encode "this pattern means trouble", and prints a report. No Spark cluster, no History Server, and no network access are required to run it - it is a pure file-in / text-out program.

> **THE ONE-SENTENCE VERSION**
> Point it at a Spark event log; it hands back a ranked list of what is slow in that job and what to try next.

1.2 What a Spark event log is

While a Spark application runs, the driver emits a stream of *listener events* - one for the application starting, one for each job, one when a stage is submitted, one when every single task ends, and so on. If event logging is enabled, Spark serialises each of those events to one line of JSON and appends it to a file. A completed application therefore leaves behind a complete, append-only, replayable transcript of its own execution.

That file format is the key architectural fact about this whole project: it is **JSON Lines** - newline-delimited JSON, one self-contained object per line. That means the file can be read one line at a time without parsing the whole thing, it can be split at any newline boundary, and it can be appended to while being read. Almost every scaling opportunity discussed in Part 6 comes back to exploiting those three properties, which the current code does not yet do.

```
{"Event":"SparkListenerApplicationStart","App Name":"daily-sales-etl"}
{"Event":"SparkListenerEnvironmentUpdate","Spark Properties":{"spark.app.id":"app-20260421-0001"}}
{"Event":"SparkListenerJobStart","Job ID":3,"Stage IDs":[10,11]}
{"Event":"SparkListenerStageSubmitted","Stage Info":{"Stage ID":10,"Stage Name":"SortMergeJoin stage",
...}}
{"Event":"SparkListenerTaskEnd","Stage ID":10,"Task Info":{"Task ID":1,"Attempt":0},"Task
Metrics":{...}}
```

Five of the eleven lines of `sample_data/mock_event_log.jsonl`, abbreviated. The tool only looks at four event types and ignores every other line it sees.

1.3 The four words you need: Application, Job, Stage, Task

Spark execution is a strict four-level hierarchy, and the tool's entire data model is built on it. Understanding this nesting is the difference between the report reading as gibberish and reading as obvious.



Level	What it is	How many	Why the tool cares
Application	One submitted Spark program, start to finish.	1 per event log	Gives the report its identity: app_id and app_name.
Job	One action - a count(), a write(), a collect(). Each action triggers a job.	A handful	Only used to label which job a stage belonged to.
Stage	A run of work with no shuffle inside it. Spark cuts a new stage every time data must be redistributed across the network.	Tens	This is the unit of diagnosis. Every finding is attached to a stage.
Task	One stage applied to one partition of the data, executed on one core of one executor.	Hundreds to millions	This is the unit of measurement. All statistics are computed over tasks.

WHY STAGE BOUNDARIES ARE WHERE THE PAIN IS

Inside a stage, tasks are independent and Spark just runs them. Between stages, Spark must **shuffle**: every executor writes intermediate data to local disk, and every executor in the next stage fetches the pieces it needs over the network. Shuffle is disk plus network plus serialisation, so it is usually the most expensive thing a job does. Four of the six rules in this tool are ultimately about shuffle.

1.4 The five failure patterns the tool looks for

Pattern	Beginner explanation	Typical real cause
Shuffle bottleneck	The stage moved an enormous amount of data across the network.	Joining or grouping before filtering; too few or too many partitions.
Data skew	Most tasks finished quickly; one or two ran far longer, so the stage waits on them.	One join key is far more common than the rest (a null key, a default customer id, one huge tenant).
Straggler tasks	The slowest task is far slower than the average task.	Skew, but also a sick executor, bad data locality, or GC thrash on one node.
Retry / failure	Tasks died and Spark re-ran them, so the work was paid for more than once.	Out-of-memory executors, lost shuffle blocks (FetchFailed), corrupt input files, spot-instance eviction.
Partition imbalance	Partitions were not the same size, so the work per task was uneven.	Skewed keys again, or an input source with wildly different file sizes.

A sixth rule, *expensive wide transformation*, is a naming heuristic rather than a measurement: it flags stages whose name mentions a join, aggregate or sort *and* which shuffled a meaningful amount of data. It exists to give the other findings a human-readable cause.

1.5 Project facts at a glance

Property	Value	Why it matters
Language / runtime	Python, requires 3.10 or newer	Uses modern syntax such as X | Y unions and dataclass slots=True.
Runtime dependencies	None - standard library only	Installs anywhere, no supply chain, no version conflicts. A deliberate and unusually strong choice.
Dev dependencies	pytest only	Test suite runs in well under a second.
Source size	520 lines across 7 modules	Small enough to read end to end in one sitting.
Test suite	3 tests, 54 lines, all passing	Thin. See Part 7 for what is not covered.
Entry point	spark-perf-analyzer = spark_perf_analyzer.cli:main	Installed as a console script by pyproject.toml.
Interfaces	One: a command-line program	No library API contract, no service, no persistence.
CI	GitHub Actions, matrix on Python 3.10 / 3.11 / 3.12	Runs pytest on every push and pull request.
Concurrency model	Single process, single thread, fully synchronous	The central constraint behind everything in Part 6.

Part 2 - The pipeline, end to end

Six stages, one direction, no loops back. Follow the arrows and you have read the codebase.

The architecture is a straight pipeline. Each stage takes the previous stage's output, transforms it, and hands it on. Nothing flows backwards and nothing is shared globally, which is why the design is easy to reason about - and, as Part 6 shows, why most of it is straightforward to scale once one specific bottleneck is removed.

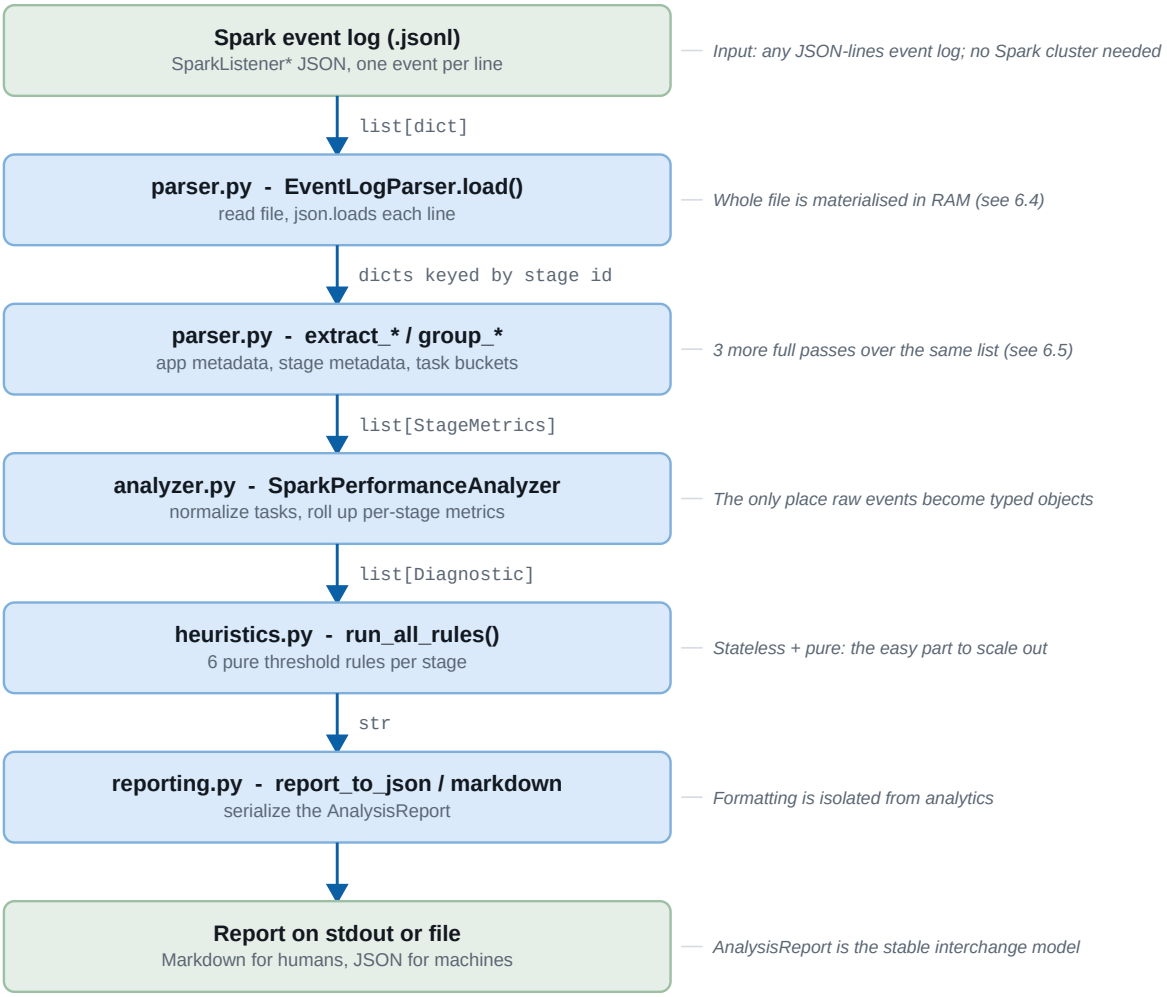


Figure 1. The full data flow. Labels on the arrows are the actual Python types handed between layers. Notes on the right preview the load behaviour analysed in Part 6.

2.1 The data contract between layers

The value of a pipeline this clean is that each boundary is a contract you can test, replace, or parallelise independently. These are the five contracts:

Boundary	Type handed across	Coupling
File to parser	A path (str or Path)	Loose. Any JSON-lines source could be substituted.
Parser to analyzer	list[dict] of raw events, plus three derived dicts	Tight and expensive - this is the whole log held in memory.
Analyzer to rules	One StageMetrics object at a time	Very loose. Rules see a flat struct of numbers and never touch raw events.
Rules to reporting	list[Diagnostic]	Very loose. Rules do not know what a report looks like.
Reporting to output	A single str, then written or printed	Loose, but materialises the entire report in memory before writing.

THE DESIGN INSIGHT WORTH TAKING AWAY

Layers 3, 4 and 5 (metrics, rules, formatting) are **pure functions over small structs**. They have no I/O, no shared state, and no ordering requirements. That means they scale horizontally for free. Essentially all of the scaling difficulty in this project lives in layer 2 - the parser - and in the fact that the analyzer holds every task of a stage in memory to compute statistics over them.

2.2 What actually executes, in order

```
cli.main()
 |
 +- build_parser().parse_args()                  # argparse: path, 2 thresholds, format, output
 +- SparkPerformanceAnalyzer(shuffle_threshold_mb=..., skew_ratio_threshold=...)
 +- analyzer.analyze(path)
 |   +- EventLogParser(path).load()              # PASS 0: file -> list[dict]  (all of it, in RAM)
 |   +- extract_application_metadata(events)     # PASS 1: find app id + app name
 |   +- extract_stage_metadata(events)           # PASS 2 and 3: StageSubmitted, then JobStart
 |   +- group_task_events(events)                # PASS 4: bucket TaskEnd events by stage id
 |   +- for each stage id:
 |   |    _build_stage_metrics(...)             # normalize every task, then roll up
 |   |      +- _normalize_task(...)  x N tasks   # dict -> TaskMetrics
 |   |      +- mean / quantiles / max            # runtime statistics
 |   |      +- _partition_imbalance_ratio(...)
 |   +- for each stage:
 |        run_all_rules(stage, ...)              # 6 pure rules -> 0..6 Diagnostic objects
 |
 +- report_to_markdown(report)  or  report_to_json(report)
 +- write_report(content, output)                # print to stdout, or write a file
```

Read top to bottom, this is the entire program. Note the five separate full traversals of the event list, marked PASS 0 through PASS 4 - that count comes back in section 6.5.

Part 3 - Module by module

What each file does, what it hands on, and how each one behaves when the input gets big.

Seven files, 520 lines. Each subsection below covers one module: its job, its public surface, the beginner-relevant details, and - the part the rest of this document builds on - what happens to that specific module when the event log grows or the request rate rises.

3.1 models.py - the vocabulary

File	Lines	Role in the pipeline
`src/spark_perf_analyzer/models.py`	61	Defines every data structure the rest of the code passes around. No logic.

Function or type	What it does	In / out
`Severity`	String enum: info / medium / high.	-
`TaskMetrics`	One task attempt: runtime, shuffle bytes in and out, input and output bytes, status.	built from one TaskEnd event
`StageMetrics`	One stage rolled up: 15 fields of counts, byte totals and runtime statistics.	the input to every rule
`Diagnostic`	One finding: rule id, title, severity, stage id, evidence, recommendation.	the output of every rule
`AnalysisReport`	app_id, app_name, list of stages, list of diagnostics.	the top-level result object

Every dataclass uses `slots=True`, which drops the per-instance `__dict__`. That is a real memory win precisely where it matters: `TaskMetrics` is instantiated once per task, so at a million tasks the choice is worth hundreds of megabytes. It is the single most load-aware decision in the current codebase.

`AnalysisReport` is described in the repo's architecture notes as the stable interchange model - the thing a future web API or database schema would be built against. Keeping it free of behaviour is what makes that plausible.

> **BEHAVIOUR AS LOAD INCREASES**
>
> Grows strictly linearly with the number of tasks and stages, and it is the cheapest possible version of that growth thanks to `slots`. The structural problem is not the size of one object, it is that **nothing ever discards them**: every TaskMetrics for a stage is alive at once so that mean, max and quantiles can be computed over the list.

3.2 parser.py - reading and normalising the log

File	Lines	Role in the pipeline
`src/spark_perf_analyzer/parser.py`	94	Turns a file on disk into raw dicts, then pulls out the four things the analyzer needs.

Function or type	What it does	In / out
`EventLogParser.load()`	Opens the file, strips blank lines, json.loads each line, appends to a list. Raises FileNotFoundError, or ValueError naming the exact bad line number.	path -> list[dict]
`extract_application_metadata()`	Scans for ApplicationStart (app name) and EnvironmentUpdate (spark.app.id). Falls back to 'unknown-app'.	list[dict] -> (str, str)
`extract_stage_metadata()`	Scans StageSubmitted for name, task count and parent ids; then scans JobStart to attach job ids.	list[dict] -> dict[int, dict]
`group_task_events()`	Buckets every TaskEnd event under its Stage ID.	list[dict] -> dict[int, list[dict]]
`_safe_int()`	Coerces anything to int, returning 0 on failure.	Any -> int

The error handling here is genuinely good for a small project: a malformed log fails immediately with the file path and the offending line number, rather than producing a silently wrong report.

`_safe_int` is the defensive spine of the parser. Spark event logs vary between versions and some metric blocks are simply absent for some task types, so every numeric read is wrapped. The cost is that a genuinely missing metric is indistinguishable from a real zero - a correctness trade-off revisited in Part 7.

```
def load(self) -> list[dict[str, Any]]:
    events: list[dict[str, Any]] = []
    with self.path.open("r", encoding="utf-8") as handle:
        for line_number, line in enumerate(handle, start=1):
            ...
            events.append(json.loads(line))     # <-- the whole log ends up here
    return events
```

parser.py:15-31. The file is read incrementally but accumulated completely; the generator that this wants to be is section 6.4.

> **BEHAVIOUR AS LOAD INCREASES**
>
> **This is the bottleneck of the entire system.** `load()` materialises the complete log as a Python list before anything else runs, so peak memory is a fixed multiple of file size - measured at roughly 5.4x in section 6.3. The repository's own architecture notes claim the log is consumed "line-by-line to support large files"; the implementation does not do that. Everything else in Part 6 is downstream of this one function.

3.3 analyzer.py - measurement and orchestration

File	Lines	Role in the pipeline
`src/spark_perf_analyzer/analyzer.py`	123	The coordinator. Calls the parser, computes per-stage statistics, invokes the rules, assembles the report.

Function or type	What it does	In / out
`SparkPerformanceAnalyzer`	Holds the two configurable thresholds. Constructed once per run.	-
`analyze()`	The single public method: path in, AnalysisReport out.	str -> AnalysisReport
`_build_stage_metrics()`	Normalises every task of one stage, then computes totals, mean, p95, max, skew ratio, imbalance ratio, and the completed / failed / retried counts.	-> StageMetrics
`_normalize_task()`	Digs the six numbers the tool needs out of a deeply nested TaskEnd event.	dict -> TaskMetrics
`_partition_imbalance_ratio()`	max(load) / mean(load) over non-zero partitions, where load is shuffle read plus input bytes. Returns 0.0 with fewer than two partitions.	-> float

The stage list is built from `sorted(set(stage_metadata) | set(task_events))`, a union rather than an intersection. That is a deliberately forgiving choice: a stage seen only in task events (its StageSubmitted line was truncated) still gets analysed, and a stage that was submitted but ran no tasks still shows up with zeros. Truncated logs are extremely common in practice, so this matters.

Percentile handling is worth understanding: `quantiles(runtimes, n=20)[18]` gives the true p95, but only when there are at least 20 task runtimes. Below that it silently falls back to `max`. So for small stages the reported p95 and max are the same number by construction - which is exactly what you see in the sample report in Part 4.

> **BEHAVIOUR AS LOAD INCREASES**
>
> Time is linear in task count and the constant is small, but memory is the issue: `_build_stage_metrics` builds a full `list[TaskMetrics]` for the stage, then derives three more lists from it (runtimes, partition loads, unique ids). A single 50,000-task stage therefore has four parallel collections alive at once. Every one of those statistics except the exact p95 could be computed in a streaming accumulator with O(1) memory - see 6.6.

3.4 heuristics.py - the rules engine

File	Lines	Role in the pipeline
`src/spark_perf_analyzer/heuristics.py`	123	Six independent detection rules plus the function that runs them all.

Function or type	What it does	In / out
`detect_shuffle_bottleneck()`	read + write bytes over the MB threshold. Severity HIGH.	StageMetrics -> Diagnostic | None
`detect_data_skew()`	runtime_skew_ratio at or above the threshold. Severity HIGH.	same
`detect_stragglers()`	max runtime at or above 2.5x the average. Severity MEDIUM.	same
`detect_failure_retry_pattern()`	retry ratio at or above 15%, or any failed task. HIGH if anything failed, otherwise MEDIUM.	same
`detect_expensive_wide_transformation()`	stage name contains join / aggregate / reducebykey / groupbykey / sort AND shuffle exceeds 128 MB. MEDIUM.	same
`detect_partition_imbalance()`	imbalance ratio at or above 2.0. MEDIUM.	same
`run_all_rules()`	Calls all six, drops the Nones, returns the survivors.	-> list[Diagnostic]

The shape of every rule is identical and worth copying: take a `StageMetrics` and a threshold, return `None` if the stage is fine, or a fully populated `Diagnostic` if it is not. Each finding carries both *evidence* (the numbers that fired the rule) and a *recommendation* (what to do about it), which is what makes the output actionable rather than merely descriptive.

These are pure functions of a small struct. No I/O, no state, no ordering. Adding a seventh rule means writing one function and adding one line to `run_all_rules`.

> **BEHAVIOUR AS LOAD INCREASES**
> Cost is O(stages x rules), which is negligible - stages number in the tens, not the millions. This layer is not a performance concern at any realistic scale. Its scaling problem is **human**: the rules overlap, so one root cause produces several findings. The sample log raises six diagnostics for what is really one skewed join. Multiply that across a thousand nightly jobs and the output stops being read. Section 6.18 treats alert fatigue as the real capacity limit here.

3.5 reporting.py - serialisation

File	Lines	Role in the pipeline
`src/spark_perf_analyzer/reporting.py`	70	Turns an AnalysisReport into text. Two formats, one writer.

Function or type	What it does	In / out
`report_to_json()`	dataclasses.asdict over stages and diagnostics; converts the Severity enum to its string value. Indented, machine-readable.	AnalysisReport -> str
`report_to_markdown()`	Builds a human report: header, per-diagnostic sections grouped by severity label, then a one-line-per-stage summary table.	AnalysisReport -> str
`write_report()`	Prints to stdout when no path is given; otherwise creates parent directories and writes UTF-8.	(str, str | None) -> None

Two formats for two audiences: Markdown pastes straight into a pull request, a ticket or a postmortem; JSON is what you pipe into `jq`, load into a dashboard, or diff between runs. Keeping both here rather than in the analyzer is why adding an HTML or CSV format later touches exactly one file.

`write_report` calling `mkdir(parents=True, exist_ok=True)` is a small courtesy that makes the tool pleasant in scripts and CI, where the output directory often does not exist yet.

> **BEHAVIOUR AS LOAD INCREASES**
> Both renderers build the entire report as one string in memory before it is written. That is fine at today's sizes - a report is proportional to the number of stages and findings, not tasks - but it is a second full copy of the output. Under an L2 service architecture (6.10) the more pressing change is that reports must be persisted and addressable rather than printed, since a report nobody stored is a report nobody can trend, diff, or alert on.

3.6 cli.py - the user interface

File	Lines	Role in the pipeline
`src/spark_perf_analyzer/cli.py`	44	argparse wiring and nothing else. Deliberately thin.

Function or type	What it does	In / out
`build_parser()`	Declares the positional event_log argument plus --shuffle-threshold-mb, --skew-ratio-threshold, --format {markdown,json} and --output.	-> ArgumentParser
`main()`	Parse args, construct the analyzer, run it, render, write.	-> None

Separating `build_parser` from `main` is what makes the CLI testable - and `tests/test_cli.py` exercises it by monkeypatching `sys.argv`. Keeping zero business logic here means the same analyzer can later be driven by a web handler or a queue worker without duplicating anything.

Two gaps stand out for the guardrail use case the README advertises. First, only two of the six rules' thresholds are reachable from the command line; the straggler multiplier, retry ratio and imbalance threshold are hard-coded defaults. Second, `main()` always exits 0, so it cannot fail a CI build on a HIGH finding, and an unreadable log surfaces as a raw Python traceback rather than a clean error message.

BEHAVIOUR AS LOAD INCREASES

The CLI itself has no load characteristics - it runs once and exits. Its significance under load is what it *implies*: one process, one log, one report, started by a human. Every capability an operations team eventually wants - scheduling, retries, concurrency, history, alerting - lives outside this interface, which is why Part 6 is largely about what surrounds the CLI rather than what is inside it.

3.7 __init__.py - the public surface

File	Lines	Role in the pipeline
`src/spark_perf_analyzer/__init__.py`	5	Re-exports SparkPerformanceAnalyzer so importers get one obvious entry point.

Five lines, but they encode a decision: the package's supported API is the analyzer class. The parser, the models and the individual rules are importable but are not advertised as stable. For a project that intends to grow a plugin registry later, that narrow surface is the right starting point.

BEHAVIOUR AS LOAD INCREASES

No load behaviour of its own. Worth noting that a future service does not need a new API here - `SparkPerformanceAnalyzer.analyze()` is already the correct unit of work for a queue consumer to call.

Part 4 - Worked example: tracing the sample log

Eleven lines of JSON, all the way to six findings, with the arithmetic shown.

The repository ships an 11-line mock event log. Running the tool against it is the fastest way to make the pipeline concrete. Everything below is the actual output of the actual code, not an illustration.

```
$ spark-perf-analyzer sample_data/mock_event_log.jsonl \
      --format markdown --shuffle-threshold-mb 256 --skew-ratio-threshold 2.0
```

4.1 What the 11 lines contain

Lines	Event	Contributes
1	SparkListenerApplicationStart	app_name = daily-sales-etl
2	SparkListenerEnvironmentUpdate	app_id = app-20260421-0001
3	SparkListenerJobStart	Job 3 owns stages 10 and 11
4	SparkListenerStageSubmitted	Stage 10 - 'SortMergeJoin stage', 4 planned tasks, parent 9
5-8	SparkListenerTaskEnd x 4	Stage 10 task attempts (one of them a retry that failed)
9	SparkListenerStageSubmitted	Stage 11 - 'mapPartitions narrow stage', 2 planned tasks
10-11	SparkListenerTaskEnd x 2	Stage 11's two successful tasks

4.2 Stage 10's four task attempts, as parsed

Task	Attempt	Runtime (ms)	Shuffle read	Shuffle write	Input	End reason
1	0	800	200 MB	60 MB	70 MB	Success
2	0	900	180 MB	70 MB	60 MB	Success
2	1	1,200	220 MB	90 MB	95 MB	FetchFailed
3	0	5,100	650 MB	250 MB	500 MB	Success

Task 2 appears twice: attempt 0 and attempt 1. That duplicate Task ID is how the tool detects a retry, and the FetchFailed reason on attempt 1 is what makes the failure count non-zero.

4.3 The roll-up arithmetic, step by step

This is exactly what `_build_stage_metrics` computes for stage 10. Every number below is reproducible from the table above.

```
runtimes            = [800, 900, 1200, 5100]

duration_ms         = sum(runtimes)                     = 8,000 ms
runtime_avg_ms      = mean(runtimes)                    = 2,000.0 ms
runtime_max_ms      = max(runtimes)                     = 5,100 ms
runtime_p95_ms      = max(runtimes)                     = 5,100 ms   <- only 4 samples, so p95 falls
back to max
runtime_skew_ratio  = 5100 / 2000                       = 2.55

shuffle_read_bytes  = 200 + 180 + 220 + 650  MB         = 1,250,000,000 B  (1.16 GiB)
```

```
shuffle_write_bytes =  60 +  70 +  90 + 250  MB          =   470,000,000 B  (0.44 GiB)

partition load      = shuffle_read + input, per task
                    = [270, 240, 315, 1150] MB
mean(load)          = 493.75 MB
partition_imbalance = 1150 / 493.75                      = 2.33

completed_tasks     = count(end reason == "Success")     = 3
failed_tasks        = count(end reason != "Success")     = 1
retried_tasks       = 4 attempts - 3 distinct task ids   = 1
num_tasks           = "Number of Tasks" from StageSubmitted = 4
```

4.4 Which rules fire, and why

Rule	Test applied	Result
shuffle_volume	1.60 GB total shuffle vs the 256 MB threshold	FIRES - HIGH
runtime_skew	skew ratio 2.55 vs the 2.0 threshold passed on the command line	FIRES - HIGH
straggler_tasks	max 5,100 ms vs avg 2,000 ms x 2.5 = 5,000 ms	FIRES - MEDIUM (barely)
retry_failure	1 failed task, so it fires regardless of the 25% retry ratio	FIRES - HIGH
wide_transformation	name contains 'sortmergejoin' AND shuffle above 128 MB	FIRES - MEDIUM
partition_imbalance	ratio 2.33 vs the 2.0 threshold	FIRES - MEDIUM

All six fire on one stage. Stage 11 - two fast tasks, 2.1 MB of shuffle, no failures - fires none, which is the correct and reassuring result. The second test in the suite proves the thresholds really do control this: raising the shuffle threshold to 4096 MB and the skew threshold to 10.0 makes both of those findings disappear.

> **READ THIS THE WAY AN INTERVIEWER WOULD**
>
> Six findings for one problem is not six problems. The skew, the stragglers and the partition imbalance are three different measurements of the *same* hot partition - task 3, which read 650 MB of shuffle against a 494 MB average and consequently ran 2.55x longer than its peers. The shuffle volume and the wide transformation findings are two views of the join that created that partition. A senior engineer reads this report as: **one skewed sort-merge join, plus a fetch failure that is probably a symptom of the same overloaded executor.** Section 6.18 is about teaching the tool to say that itself.

Part 5 - The six rules, decoded

Exact trigger conditions, defaults, and what each finding means operationally.

Rule id	Fires when	Default	Severity	Tunable from CLI?
shuffle_volume	shuffle_read + shuffle_write >= threshold	512 MB	HIGH	Yes --shuffle-threshold-mb
runtime_skew	runtime_max / runtime_avg >= threshold	3.0	HIGH	Yes --skew-ratio-threshold
straggler_tasks	runtime_max >= runtime_avg x multiplier	2.5	MEDIUM	No - hard-coded
retry_failure	retried / num_tasks >= ratio, OR any failed task	0.15	HIGH if failures, else MEDIUM	No - hard-coded
wide_transformation	stage name matches a wide-op keyword AND shuffle >= 128 MB	128 MB	MEDIUM	No - hard-coded
partition_imbalance	max(load) / mean(load) >= threshold	2.0	MEDIUM	No - hard-coded

A STRUCTURAL OVERLAP WORTH KNOWING ABOUT

`runtime_skew` fires when max/avg is at or above 3.0 by default. `straggler_tasks` fires when max/avg is at or above 2.5. The second condition is strictly weaker than the first, so **every stage flagged for runtime skew is also, always, flagged as a straggler** - guaranteed duplication by construction, not bad luck. The two rules are measuring the identical quantity against two thresholds. Merging them into one rule with graded severity would lose no information and cut the noise.

5.1 What each finding means, and what to actually do

Finding	What it is telling you	First things to try
High shuffle volume	This stage pushed a large amount of data across the network and through local disk. Shuffle is usually the dominant cost in a Spark job, so a large number here is where the time went.	Filter and project before the join, not after. Check spark.sql.shuffle.partitions - the 200 default is wrong in both directions for most real workloads. Consider a broadcast join if one side is small.
Task runtime skew	The stage finished when its slowest task finished. If one task took 2.5x the average, most of your cluster sat idle waiting for it.	Find the hot key. Enable Adaptive Query Execution skew join handling (spark.sql.adaptive.skewJoin.enabled). Salt the heavy key. Filter out null or sentinel join keys.
Straggler tasks	Same measurement as skew, at a lower threshold. If it fires but skew does not, suspect the environment rather than the data.	Check whether the slow tasks all landed on one executor or host. Look at GC time and at data locality (NODE_LOCAL vs ANY).
Retry / failure pattern	Work was done more than once and paid for more than once. FetchFailed in particular means shuffle blocks went missing - usually an executor died.	Read executor logs for OOM kills. Raise executor memory or lower cores per executor. On spot or preemptible instances, enable external shuffle service or decommissioning.
Expensive wide transformation	A named, known-costly operation (join, aggregate, sort) that also moved real data. This is the explanatory finding - it names the operation the other rules are complaining about.	Re-examine the join strategy. Broadcast the small side if it fits. Pre-aggregate before joining. Reduce the number of columns carried through the shuffle.
Partition imbalance	Bytes per partition were uneven, so work per task was uneven, so the stage was rate-limited by its fattest partition.	Repartition on a higher-cardinality key. Tune the shuffle partition count. Let AQE coalesce small partitions post-shuffle.

Part 6 - System design: what happens as load increases

The core question. Measured on this machine against generated logs of up to 1.77 GB.

Everything so far described a program that works. This part asks the harder question: what breaks, in what order, as the input gets bigger and the request rate goes up - and what the architecture should become at each step. Every number in sections 6.3 through 6.7 was measured by running this repository's code, not estimated.

6.1 Naming what we have today

Dimension	Current design	Consequence under load
Execution model	One synchronous process, invoked by a human	No parallelism, no queueing, no retry
State	None. Nothing is persisted between runs	No history, so no regression detection
Input handling	One local file path	Cannot reach cloud object storage or compressed logs
Memory model	Whole log materialised as Python objects	Peak RSS is a fixed multiple of file size
Failure model	Exception propagates and the process dies	One bad byte discards the entire analysis
Output	Text to stdout or one file	Results are not addressable, queryable or diffable
Scaling knob	Buy a bigger machine	Vertical only, and it runs out

None of this is a criticism of the MVP - a zero-dependency, single-file CLI is the correct shape for proving that the diagnostics are useful. It is simply the honest starting point, and naming it precisely is what makes the next steps obvious.

6.2 The five independent load dimensions

"More load" is not one thing. These five axes fail differently and are fixed differently, and conflating them is the most common mistake in a scaling discussion.

Axis	From	To	What breaks first
Log size (vertical)	9 MB toy log	10-100 GB from a long-running or streaming app	Memory. The process is OOM-killed.
Log count (horizontal)	A few by hand	5,000+ applications per night	Throughput and scheduling. There is no queue and no worker pool.
Concurrent users	One engineer	A whole platform team plus CI plus dashboards	There is no service to be concurrent against.
Rule count	6 rules	Dozens, versioned, per-team profiles	Not compute - human attention. Findings stop being read.
History depth	None	90 days of trend and regression detection	Storage and schema. Nothing is persisted at all today.

6.3 Measured scaling behaviour of the current code

Synthetic event logs shaped like real ones (full Task Info and Task Metrics blocks, roughly 880 bytes per task) were generated at four sizes and analysed in a fresh subprocess each time. Peak RSS is the process high-water mark from `getrusage`.

Tasks	Log size	Wall time	Peak RSS	Throughput	RSS / log size
10,000	8.8 MB	0.51 s	61 MB	17.2 MB/s	6.9x
100,000	88.0 MB	5.00 s	487 MB	17.6 MB/s	5.5x
500,000	441.0 MB	28.16 s	2,381 MB	15.7 MB/s	5.4x
2,000,000	1,767.6 MB	101.91 s	9,479 MB	17.3 MB/s	5.4x

Table 1. Current implementation. Both time and memory are cleanly linear in input size - the algorithm is O(N) with no hidden quadratic term. The problem is the constant on the memory axis.

Two numbers matter. **Throughput is flat at roughly 17 MB/s**, which is respectable for pure-Python JSON parsing and is not the limiting factor. **Peak RSS settles at 5.4x the log size**, and that is the number that ends the story: it converts a disk-space problem into a memory problem, and memory is the resource you cannot stream.

> **THE MEMORY WALL, STATED AS A RULE OF THUMB**
>
> **Largest analysable log is roughly (available RAM) / 5.4.** On a 16 GB laptop that is about a 3 GB event log; on an 8 GB container, about 1.5 GB. Beyond it the process is not slow - it is killed. While preparing this document, an in-process run at 2 million tasks was terminated by the OOM killer with exit code 137, which is precisely the failure mode described here.

6.4 Bottleneck 1: the log is fully materialised (memory)

`EventLogParser.load()` reads the file incrementally but appends every parsed event to a list, returning only once the whole log is resident. Every downstream function then takes that list. The repository's own `docs/ARCHITECTURE.md` states the intent - "consume Spark event logs line-by-line to support large files" - so this is a gap between the documented design and the implementation, not a disagreement about what is right.

The fix is small and does not change any caller's logic, only its type:

```
# now:  builds the whole log in RAM, then returns it
def load(self) -> list[dict[str, Any]]:
    events = []
    for line_number, line in enumerate(handle, start=1):
        events.append(json.loads(line))
    return events

# streaming:  yields one event at a time, peak memory is one line
def iter_events(self) -> Iterator[dict[str, Any]]:
    with self.path.open("r", encoding="utf-8") as handle:
        for line_number, line in enumerate(handle, start=1):
            line = line.strip()
            if not line:
                continue
            try:
                yield json.loads(line)
            except json.JSONDecodeError:
                self.malformed_lines += 1        # count it, report it, keep going
```

Note the second change folded in: skipping and counting a malformed line instead of raising. Section 6.13 explains why all-or-nothing parsing is the wrong policy once logs are large and arrive unattended.

6.5 Bottleneck 2: five passes over the same data (CPU)

`analyze()` traverses the event collection five times: once to build the list, once for application metadata, twice inside `extract_stage_metadata` (stage submissions, then job starts), and once to bucket task events. Because all five look at the same immutable stream, they can be fused into a single pass with one dispatch on `event["Event"]`.

That fusion is what makes streaming possible at all: you cannot iterate a generator five times. The two fixes are therefore one fix, and they must be done together.

6.6 Bottleneck 3: every task of a stage is held to compute statistics

`_build_stage_metrics` builds a full `list[TaskMetrics]` and then derives three more collections from it. But look at what is actually needed: sum, mean, max, count, and a max-over-mean ratio. **Every one of those is computable in constant memory from running accumulators.** Only the exact 95th percentile genuinely needs the distribution - and it can be approximated to within about 1% by a t-digest or reservoir sample in a few kilobytes.

Statistic	Needs	Streaming form
duration_ms, shuffle totals	a running sum	O(1)
runtime_avg_ms	sum and count	O(1)
runtime_max_ms, skew ratio	a running max, plus the mean	O(1)
partition_imbalance_ratio	running max and running mean of load	O(1)
completed / failed counts	two counters	O(1)
retried_tasks	a set of seen ids today; a counter on (Index, Attempt) instead	O(1)
runtime_p95_ms	the distribution	t-digest, a few KB, ~1% error

Measured result of applying 6.4, 6.5 and 6.6

A prototype implementing exactly these three changes - iterator parsing, one fused pass, per-stage accumulators - was run against the same 1.77 GB, 2-million-task log:

Implementation	Wall time	Peak RSS	Memory vs log size	Improvement
Current (materialises the log)	127.4 s	9,480 MB	5.4x	baseline
Streaming, exact statistics retained	20.5 s	204 MB	0.12x	46x less memory, 6.2x faster
Streaming, pure O(1) accumulators	14.9 s	10 MB	constant	948x less memory, 8.5x faster

Table 2. Same machine, same input, same diagnostics. The last row's memory does not grow with the log at all - it is O(number of stages), which is in the tens.

THIS IS THE SINGLE HIGHEST-LEVERAGE CHANGE IN THE PROJECT

Ten megabytes, flat, regardless of input size. The memory wall from 6.3 disappears entirely: a 100 GB event log becomes analysable on a laptop, and it gets faster at the same time because five passes became one and millions of short-lived objects are never allocated. Roughly 60 lines of the parser and analyzer change; the models, the rules, the reporting and the CLI are untouched. That is the payoff of the clean layering described in Part 2.

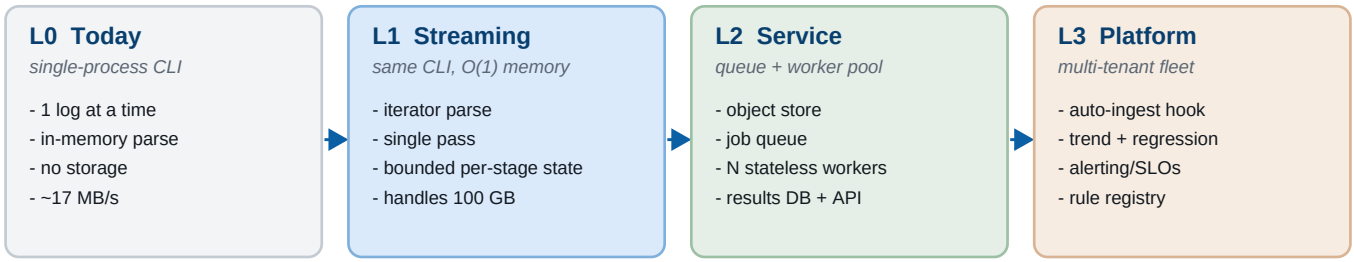
6.7 Bottleneck 4: no concurrency, and nowhere to put it

The rules layer is embarrassingly parallel - six pure functions over independent stages - but there is nothing to parallelise, because rule evaluation is a rounding error next to parsing. Within a single log, the useful concurrency is I/O and parsing, and Python's GIL makes threads unhelpful for the `json.loads` hot path.

So the honest answer is that intra-log parallelism is the wrong tool. In priority order:

1. **Fix the constant first.** The streaming rewrite gives 8.5x for zero concurrency complexity. Always take the free win before adding coordination.
2. **Parallelise across logs, not within one.** Analysing 5,000 independent logs is trivially shardable - one process per log, no shared state, no coordination. This is where the throughput is.
3. **Only then, split a single huge log.** JSON Lines can be split at any newline, so a multi-gigabyte log can be range-split across processes and the per-stage accumulators merged, because sum, max and count are all associative. Worth doing only for the p99 log that must be analysed quickly on its own.

6.8 The scaling ladder

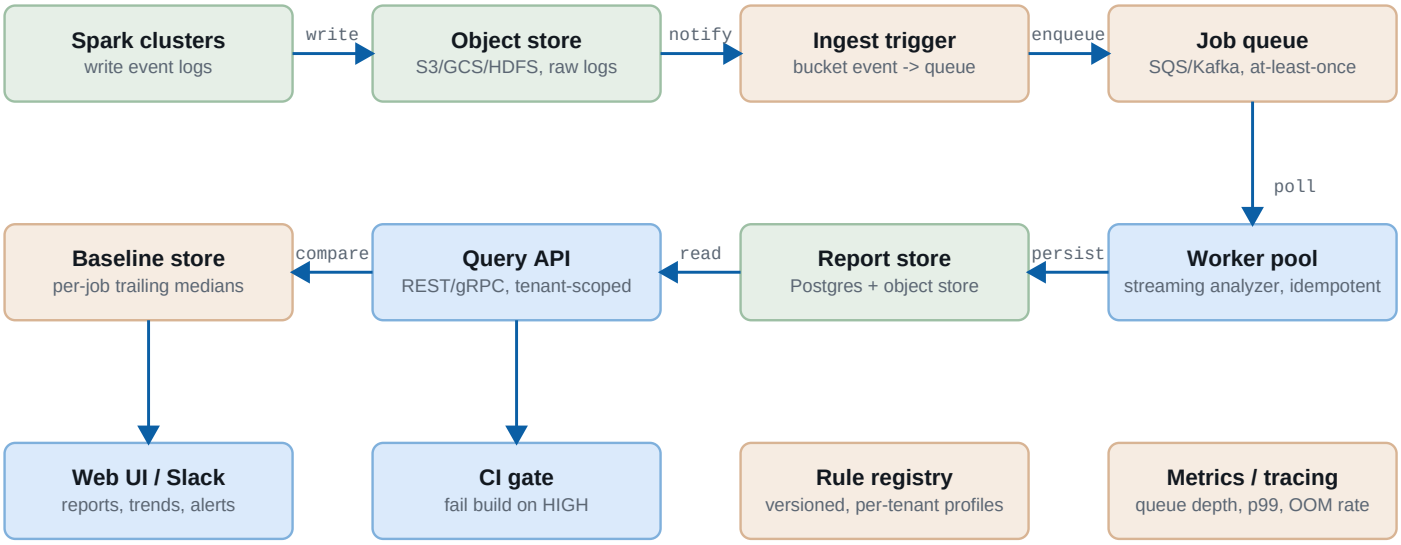


Each level is a superset of the one to its left; the code from L1 is what L2 workers run.

Figure 2. Four levels of maturity. Most teams need L1, and only a platform team serving many teams needs L2 or L3. Deciding which level you are actually at prevents both under- and over-engineering.

Level	Trigger to move up	What you build	What it costs you
L0 - CLI	-	What exists today	Nothing. Correct choice for validating the diagnostics.
L1 - Streaming CLI	Your first OOM, or a log over about 1 GB	Sections 6.4-6.6, plus gzip/zstd support and object-store paths	About 60 lines. No new infrastructure, no operational burden.
L2 - Service	You want every job analysed automatically, not on demand	Object store, queue, stateless worker pool, report database, read API	Real infrastructure and an on-call rotation. Do not do this before L1.
L3 - Platform	Multiple teams, trend and regression detection, alerting	Rule registry with versioning, per-tenant profiles, historical baselines, notification routing	The rules become a product surface with compatibility obligations.

6.9 Target architecture at L2 and L3



Blue = code that already exists in this repo (the analyzer core is the worker). Orange = new infrastructure introduced at L2/L3.

Figure 3. The analyzer core from this repository becomes the worker. Everything else is infrastructure that surrounds it without modifying it.

The most important property of this design is what it does *not* change. `SparkPerformanceAnalyzer.analyze()` is already the right unit of work for a queue consumer: it takes an input reference, does bounded work, and returns a serialisable result with no side effects. The service is built around the existing code, not instead of it.

Component	Responsibility	Key design decision
Object store	Holds raw event logs; Spark already writes here	Never copy logs into the service. Workers stream them by range request.
Ingest trigger	Bucket notification turns a new log into a queued job	Filter out .inprogress suffixes so running applications are not analysed prematurely.
Job queue	Decouples arrival rate from processing rate; absorbs the nightly burst	At-least-once delivery, so the worker must be idempotent (6.12).
Worker pool	Runs the streaming analyzer; stateless and horizontally scalable	Size by peak, not average (6.11). Route oversized logs to a large-memory pool.
Report store	Small structured results, queryable and retained	Reports are roughly 10,000x smaller than logs, so keep them far longer.
Query API	Serves reports, trends and comparisons to UI, CI and alerting	Tenant scoping enforced here, not in the UI.
Rule registry	Versioned rule definitions and per-team threshold profiles	Stamp the rule-set version into every report, or trends become meaningless (6.17).

6.10 Capacity planning: sizing the worker pool

A concrete scenario makes the arithmetic real. Assume a platform running **5,000 Spark applications per day** with a **median event log of 200 MB**, and - as is true of almost every batch platform - roughly **70% of those finish inside a two-hour nightly window**.

Quantity	Current code (~15 MB/s)	Streaming (~119 MB/s)
Time to analyse one 200 MB log	13.3 s	1.7 s
Total daily compute	66,700 s (18.5 core-hours)	8,400 s (2.3 core-hours)
Average concurrency over 24 h	0.8 workers	0.1 workers
Work arriving in the 2 h night window	46,550 s	5,950 s
Workers to merely keep up at peak	6.5	0.8
Workers provisioned for headroom (2x)	13	2
RAM required per worker	16 GB (5.4x a 3 GB p99 log)	1 GB (flat)

Table 3. Applying Little's Law (L = arrival rate x service time) to the burst window rather than the daily average.

> **THE LESSON HIDING IN TABLE 3**
>
> Sizing on the *daily average* suggests one worker is plenty. Sizing on the *burst* demands thirteen. Batch data platforms are among the burstiest workloads there are - everything is scheduled at midnight - so peak-to-average ratios of 8x or more are normal. Always provision against the burst, and note that the streaming rewrite from 6.6 shrinks the fleet from 13 large-memory machines to 2 small ones. **The algorithmic fix and the capacity plan are the same conversation.**

6.11 Backpressure and the heavy tail

Event log sizes are heavy-tailed. A median of 200 MB routinely coexists with a p99 of several gigabytes, produced by long-running streaming applications or jobs with millions of tasks. Under the current 5.4x memory factor, a single 5 GB log demands 27 GB of RAM and will kill any normally sized worker - and because the queue redelivers on failure, it will then kill the next worker, and the next. **One pathological input becomes a fleet-wide outage.**

- **Bound the queue and shed load.** An unbounded queue converts a throughput problem into an out-of-memory problem somewhere else. Prefer visible rejection over invisible unbounded growth.
- **Route by size.** Read the object's content-length at enqueue time and send large logs to a large-memory pool, or to the range-splitting path from 6.7. Cheap, and it protects the common case.
- **Cap attempts and dead-letter.** After two or three failures a message goes to a dead-letter queue for a human. This is what stops the poison-pill cascade described above.
- **Set a hard per-job budget.** Wall-clock and memory limits per worker, so a pathological log fails one job rather than an availability target.
- **Prefer degraded results over none.** A report covering the first 10 million events with a clear "truncated" flag is far more useful than an OOM and no report at all.

6.12 Idempotency and caching - and why they are free here

At-least-once queues mean every job will occasionally run twice. The natural defence is a deterministic identity for each unit of work, and this domain hands you one: **a completed Spark event log is immutable.** Spark never rewrites a finished log. That single fact makes the whole caching and idempotency story trivial, where in most systems it is the hard part.

- **Idempotency key** = content hash of the log (or the object store's ETag) combined with the rule-set version. Writing a report becomes an upsert on that key, so duplicate delivery is harmless.
- **Cache lookups are always safe.** If a report already exists for that key, the answer cannot have changed - there is no invalidation problem because there is no mutation.
- **Reprocessing is deliberate and cheap.** Ship a new rule version and the key changes, so exactly the logs you want re-analysed are re-analysed, and the old reports remain valid for their own version.
- **Retries are safe by construction.** The analyzer is a pure function of the log; a failed job can simply be re-run with no compensating action or cleanup.

6.13 Failure handling: all-or-nothing parsing does not survive contact with reality

Three failure modes were reproduced against this repository's code. All three are common in production, and all three currently destroy an entire analysis:

Situation	How common	Current behaviour	What it should do
Truncated final line - a running (.inprogress) application, or a killed driver	Very common	ValueError; no report at all	Analyse everything up to the truncation, flag the report as partial
One corrupt line in the middle of an otherwise good multi-gigabyte log	Occasional	ValueError; the whole log is discarded	Skip the line, increment a malformed_lines counter, surface it in the report
Compressed log (.gz, .lz4, .zstd, .snappy) - Spark's common configuration	Very common	UnicodeDecodeError, not even a clear message	Sniff the magic bytes and pick the decompressor transparently

The underlying principle is the shift from **fail-fast** to **degrade-and-report**. Fail-fast is right for an interactive tool a human just launched: the error is immediate, visible, and the human retries. It is wrong for an unattended pipeline, where the operator is not watching, the input is not reproducible, and 99.99% of a log is still worth analysing. The current parser's precise error message - with the file path and exact line number - is genuinely good engineering; the change needed is not to detect less, but to *continue* after detecting.

6.14 Multi-tenancy and security

This is the factor most easily forgotten, and the one with the worst downside. **Spark event logs are sensitive data.** They routinely contain:

- **Table names, file paths and bucket names** in stage names and physical plan strings - a map of the organisation's data estate.
- **SQL fragments with literal values** baked into plan descriptions, which can embed customer identifiers or other personal data.
- **Hostnames, executor and container ids** - internal infrastructure topology.
- **The entire Spark configuration**, in the SparkListenerEnvironmentUpdate event's Spark Properties block. This is the dangerous one: that map can contain cloud storage keys, JDBC passwords and tokens. Spark ships spark.redaction.regex specifically because of it.

> **A LATENT RISK IN THIS CODE THAT IS WORTH NAMING NOW**
>
> `extract_application_metadata` already reads the Spark Properties map - but deliberately takes only `spark.app.id` from it, so nothing sensitive reaches the report today. That safety is incidental, not enforced. The moment someone adds a "show the configuration in the report" feature - an obvious and reasonable request - credentials start appearing in Markdown files that get pasted into tickets and chat. **Redact on ingest, allowlist the properties you extract, and never echo that map wholesale.**

Beyond redaction, a multi-tenant deployment needs the usual controls, enforced server-side:

- Every report carries a tenant id, and the query API filters on the caller's identity - never on a parameter the client supplies.
- Workers read logs with per-tenant scoped credentials, so a bug cannot read another team's bucket.
- Retention and deletion are per-tenant, because deletion requests apply to raw logs too.
- Rule profiles are per-tenant: a threshold that means "broken" for a latency-sensitive service is unremarkable for a nightly backfill.

6.15 Storage, retention and the economics of extraction

At 5,000 applications per day and a 200 MB median, raw logs accrue at about **1 TB per day**. Reports, by contrast, are a few tens of kilobytes each - roughly **100 MB per day** for the same workload. That is a ratio of about 10,000 to 1, and it dictates the entire storage policy.

Tier	Contents	Retention	Rationale
Hot	Reports and derived metrics	Indefinite - years	Tiny, and this is what trends are built from
Warm	Raw event logs, recent	7 to 30 days	Long enough to re-analyse under a new rule version or debug by hand
Cold / archive	Raw logs for flagged or incident-related runs only	90+ days	Selective, so cost stays bounded
Deleted	Everything else	-	1 TB per day compounds fast

This is the general principle that the tool already embodies without saying so: **extract the small structured summary early, then let the bulk data expire.** The analyzer turns 200 MB of transcript into 20 KB of answers. Designing the retention policy around that ratio is what keeps a diagnostics platform from costing more to run than the jobs it diagnoses.

6.16 Observability and service levels

A diagnostics tool that is itself undiagnosed is an unpleasant irony. The instrumentation worth having from day one of L2:

Signal	Why it is the one to watch
Queue depth and age of the oldest message	The single best leading indicator. Depth rising steadily means arrival rate has overtaken service rate; you need workers before anyone notices.
Analysis latency, p50 / p95 / p99	The heavy tail from 6.11 shows up here first. A p99 far above p50 means large logs need their own pool.
Worker OOM-kill rate	Directly measures the memory wall from 6.3. Should be zero after the L1 rewrite.
Malformed-line rate and partial-report rate	A jump usually means a Spark version upgrade changed the event schema underneath you.
Bytes processed per second per worker	Detects throughput regressions and tells you honestly when to scale out.
Findings per report, by rule	The alert-fatigue metric. A rule firing on 90% of stages is not a signal, it is a threshold that needs recalibrating.

A reasonable starting service level: **99% of completed applications have a report available within 15 minutes of the log being written**, with a separate, looser target for logs above some size threshold. Stating it that way makes the size-based routing from 6.11 an obvious requirement rather than an optimisation.

6.17 History: the feature that changes what the tool can say

Everything so far treats each log in isolation, and that caps how useful any finding can be. A report saying "this stage shuffled 1.6 GB" leaves the reader asking the only question that matters: *is that normal?* Persisting reports - which costs almost nothing, per 6.15 - lets the tool answer questions it structurally cannot answer today:

- **Regression detection:** this job's stage 10 shuffled 1.6 GB tonight against a 400 MB trailing median. That is a far stronger signal than any fixed threshold, because it is calibrated per job.
- **Adaptive thresholds:** replace globally hard-coded constants with per-job percentiles. This is the principled answer to the tuning problem, since one threshold cannot fit both a 30-second job and a 6-hour backfill.
- **Change attribution:** correlate the first bad run with a deployment or a configuration change.
- **Fleet-level prioritisation:** rank all jobs by wasted compute so the platform team fixes the expensive problems rather than the loudest ones.

> **THE VERSIONING OBLIGATION THAT COMES WITH HISTORY**
>
> The moment reports are retained and compared over time, **the rule set becomes part of the data model**. Change a threshold and every historical comparison silently shifts - a job appears to regress when only the ruler changed. Stamp the rule-set version into every stored report, and treat a threshold change as a schema migration, not a config tweak. This obligation does not exist at L0 and is easy to miss on the way to L3.

6.18 The real capacity limit at scale is human attention

Part 4 showed six diagnostics raised for what is genuinely one problem: a skewed sort-merge join. At one job that is mild redundancy. Across 5,000 nightly jobs it is **tens of thousands of findings per night**, and a report nobody reads has zero value no matter how fast it was produced. Compute scales by adding workers; attention does not scale at all.

Technique	What it does	Applied to the Part 4 example
Merge overlapping rules	runtime_skew and straggler_tasks measure the identical max/avg quantity against two thresholds	6 findings become 5
Correlate to a root cause	Group findings that share a stage and a causal story into one finding with supporting evidence	5 become 1: 'skewed join on stage 10', with the rest as evidence
Rank by impact, not severity	Order by estimated wasted compute - a HIGH on a 3-second stage matters less than a MEDIUM on a 3-hour one	Stage 11 never surfaces at all
Cap the report	Top N findings per application, with the rest available on demand	Bounded reading time regardless of job complexity
Suppress the known	Let teams acknowledge an accepted finding so it stops resurfacing nightly	Chronic known-skew jobs go quiet
Alert on change, not state	Notify when a job's profile shifts, not every time a threshold is crossed	5,000 nightly reports become a handful of notifications

6.19 What breaks first, in order

A consolidated answer to the question this part opened with:

#	Breaks at	Symptom	Fix	Section
1	A log above roughly (RAM / 5.4)	OOM kill, exit 137, no report	Streaming parser, fused pass, O(1) accumulators	6.4-6.6
2	Any truncated, corrupt or compressed log	ValueError or UnicodeDecodeError; whole run lost	Degrade-and-report; sniff compression	6.13
3	More logs than a human will launch by hand	Coverage silently drops to whatever someone remembered	Ingest trigger, queue, stateless worker pool	6.9
4	The nightly burst	Reports arrive hours late even though daily average looks fine	Size the pool against the burst, not the mean	6.10
5	The first multi-gigabyte outlier	One poison log repeatedly kills workers	Size-based routing, attempt caps, dead-letter queue	6.11
6	The first person who wants yesterday's numbers	No history exists; nothing can be compared	Persist reports; version the rule set	6.17
7	Roughly a few hundred reports a night	Output is ignored; the tool quietly stops being used	Correlate, rank by impact, alert on change	6.18

THE THROUGH-LINE

Failures 1 and 2 are **bugs in this repository** and are fixed by changing about 60 lines. Failures 3 through 7 are **missing systems** around the repository, and no amount of editing these seven modules will address them. Knowing which kind of problem you are looking at is the whole skill.

Part 7 - Correctness gaps found while writing this document

Reproduced against the code in this repository. Included so the guide is useful rather than promotional.

The two HIGH items below are the ones to care about, because they do not show up against the bundled sample log - they appear only against event logs that real Spark actually writes. Both were reproduced.

7.1 HIGH - task status is misread on real Spark logs

`_normalize_task` reads the end reason as `str(event.get("Task End Reason", "Unknown"))` and compares it to the literal `"Success"`. The bundled mock log stores that field as a plain string, so the tests pass. Real Spark writes it as a JSON *object* - `{"Reason": "Success"}` - and stringifying a dict never equals `"Success"`. Every task is therefore counted as failed:

```
$ # a two-task stage, both tasks successful, in real Spark's format
$ cat realshape.jsonl
{"Event":"SparkListenerTaskEnd","Stage ID":1, ... ,"Task End Reason":{"Reason":"Success"}}
{"Event":"SparkListenerTaskEnd","Stage ID":1, ... ,"Task End Reason":{"Reason":"Success"}}

  ->  completed_tasks = 0    failed_tasks = 2
  ->  [HIGH] retry_failure : failed_tasks=2, retried_tasks=0, retry_ratio=0.00%
```

Actual output. A healthy stage produces a false HIGH finding, and the failure counts in every report are wrong. The fix is two lines: take `reason["Reason"]` when the field is a dict.

7.2 HIGH - retry detection does not work on real Spark logs

`retried_tasks` is computed as the number of task events minus the number of distinct Task IDs, so it detects a retry by a repeated Task ID. That is how the mock log models a retry. Real Spark assigns a **new, monotonically increasing Task ID to every attempt**; the retry is identifiable by a repeated `Index` (the partition) with an incremented `Attempt`. Against a real log this expression is always zero, so the retry half of `detect_failure_retry_pattern` never fires. Note that `_normalize_task` already parses `Attempt` - it is simply never used.

7.3 Everything else, ranked

Sev	Issue	Detail	Where
MED	One bad line discards the whole log	Reproduced: a corrupt line at line 6 of an 11-line log aborts the entire analysis. Also applies to the truncated final line of any .inprogress log.	parser.py:25-30
MED	Compressed logs unsupported	A .gz log fails with UnicodeDecodeError rather than a clear message. Spark commonly compresses event logs; the README already lists this as a good first issue.	parser.py:20
MED	duration_ms is not a duration	It is the sum of task execution times, so it exceeds wall-clock duration by roughly the parallelism factor. Stage Info carries Submission Time and Completion Time, which are not read.	analyzer.py:83
MED	p95 is silently the max below 20 tasks	quantiles() needs 20 samples, so smaller stages report max under a field named p95. A JSON consumer cannot tell which one it received.	analyzer.py:64
MED	Stage attempts are collapsed	Stage metadata is keyed on Stage ID alone, so a resubmitted stage overwrites the first and their task events merge into one bucket. Stage Attempt ID is not read.	parser.py:64
MED	Cannot gate CI on findings	main() always exits 0, so the guardrail use case the README advertises is not achievable. A --fail-on-severity flag would close this.	cli.py:26-40
MED	Two rules are redundant by construction	runtime_skew (max/avg >= 3.0) is a strict subset of straggler_tasks (max/avg >= 2.5). Every skew finding is always accompanied by a straggler finding.	heuristics.py:27-55
LOW	Four of six thresholds are unreachable	Only the shuffle and skew thresholds are exposed on the CLI; the straggler multiplier, retry ratio and imbalance threshold are hard-coded defaults.	cli.py:14-15
LOW	_safe_int is duplicated	Defined identically in both modules. Should live in one place.	parser.py:34, analyzer.py:16
LOW	Docs contradict the implementation	ARCHITECTURE.md states logs are consumed line-by-line to support large files; load() materialises the entire log.	docs/ARCHITECTURE.md
LOW	Test suite is thin	3 tests, all against the single bundled mock log. No test covers malformed input, an empty log, compressed input, the JSON renderer, or any rule in isolation.	tests/

WHY 7.1 AND 7.2 MATTER MORE THAN THEIR LINE COUNTS SUGGEST

Both bugs are invisible to the test suite because the test suite validates against a mock log that models these two fields differently from real Spark. That is the classic fixture-drift trap: the tests pass, coverage looks fine, and the tool is wrong on every real input. The highest-value single addition to this repository would be **one genuine Spark event log as a test fixture** - it would have caught both.

Part 8 - Running it, extending it, and a glossary

8.1 Getting it running

```
# install (editable, with the test dependency)
python -m pip install -e ".[dev]"

# human-readable report to the terminal
spark-perf-analyzer sample_data/mock_event_log.jsonl

# machine-readable, written to a file, with custom thresholds
spark-perf-analyzer sample_data/mock_event_log.jsonl \
    --format json \
    --shuffle-threshold-mb 256 \
    --skew-ratio-threshold 2.0 \
    --output reports/today.json

# run the test suite  (3 tests, well under a second)
python -m pytest
```

Flag	Default	Meaning
`event_log (positional)`	required	Path to a JSON-lines Spark event log
`--shuffle-threshold-mb`	512	Total shuffle above this raises a HIGH shuffle_volume finding
`--skew-ratio-threshold`	3.0	max/avg task runtime above this raises a HIGH runtime_skew finding
`--format`	markdown	markdown for humans, json for tooling
`--output`	stdout	Where to write; parent directories are created automatically

8.2 Adding a seventh rule

The rules layer is the easiest place to contribute, and the pattern is fixed. Write a pure function, register it in one line:

```
# 1. heuristics.py  -  add the rule
def detect_gc_pressure(stage: StageMetrics, threshold: float = 0.15) -> Diagnostic | None:
    if stage.gc_ratio < threshold:              # needs a new field on StageMetrics
        return None
    return Diagnostic(
        rule_id="gc_pressure",
        title="Excessive GC time",
        severity=Severity.MEDIUM,
        stage_id=stage.stage_id,
        evidence=f"JVM GC accounted for {stage.gc_ratio:.0%} of executor run time.",
        recommendation="Raise executor memory, lower cores per executor, or reduce object churn.",
    )

# 2. heuristics.py  -  add one line to run_all_rules
candidates = [ ..., detect_gc_pressure(stage) ]
```

A new *metric* is slightly more work, because it crosses three files: add the field to `StageMetrics` in `models.py`, populate it in `_build_stage_metrics`, and read whatever raw field it needs in `_normalize_task`. The Markdown and JSON renderers pick up new `Diagnostic` objects automatically, and JSON picks up new `StageMetrics` fields automatically too, since it uses `asdict`.

8.3 A suggested order of work

Priority	Change	Why now	Rough size
1	Fix the Task End Reason and retry-attempt parsing (7.1, 7.2)	The tool reports wrong numbers on every real Spark log	~10 lines
2	Add a real Spark event log as a test fixture	Would have caught both of the above, and prevents the next one	1 file + 1 test
3	Streaming parser, single fused pass, O(1) accumulators (6.4-6.6)	Removes the memory wall entirely; measured 948x memory and 8.5x speed	~60 lines
4	Degrade-and-report on malformed lines; transparent decompression (6.13)	Truncated and compressed logs are the common case, not the exception	~25 lines
5	Add --fail-on-severity and a non-zero exit code	Unlocks the CI guardrail use case the README already claims	~10 lines
6	Merge the skew and straggler rules; rank findings by impact (6.18)	Cuts noise before anyone builds a fleet-wide deployment on top of it	~30 lines
7	Persist reports and add historical comparison (6.17)	Changes what the tool can say, from 'this is big' to 'this is new'	A new component

Items 1 and 2 are correctness and should precede everything. Item 3 is the highest-leverage engineering change in the project. Items 4 through 6 are what make it deployable unattended. Item 7 is where it stops being a tool and starts being a platform - and, notably, it is the only entry that requires infrastructure rather than edits to these seven files.

8.4 Glossary

Term	Meaning
AQE (Adaptive Query Execution)	Spark's runtime re-optimiser. It can coalesce shuffle partitions and split skewed join partitions using statistics observed mid-query - the built-in first response to several findings in this tool.
Backpressure	Deliberately slowing or rejecting incoming work so a system degrades predictably instead of collapsing.
Dead-letter queue	Where a message goes after failing too many times, so one poison input cannot block or crash the pipeline forever.
Event log	The append-only JSON-lines transcript Spark writes describing its own execution. The sole input to this tool.
Executor	A JVM process on a worker node that runs tasks and holds cached data.
Idempotency	The property that doing an operation twice is indistinguishable from doing it once - essential under at-least-once delivery.
JSON Lines (.jsonl)	One complete JSON object per line. Splittable at any newline, streamable, appendable. The format property that makes every optimisation in Part 6 possible.
Little's Law	L = arrival rate x service time. Used in 6.10 to size the worker pool against the nightly burst.
Materialise	To build a whole collection in memory rather than streaming it. The root cause of the memory wall in 6.3.
Shuffle	Redistributing data across the network between stages so that rows sharing a key land together. Usually the dominant cost in a Spark job.
Skew	Uneven distribution of data across partitions, so some tasks do far more work than others.
Stage	A unit of Spark execution containing no shuffle. The unit of diagnosis in this tool.
Straggler	A task far slower than its peers, holding up completion of the whole stage.
t-digest	A compact sketch that estimates percentiles in a few kilobytes regardless of how many values it has seen. The streaming replacement for exact p95 in 6.6.
Task	A stage applied to one partition, on one executor core. The unit of measurement in this tool.
Wide transformation	An operation requiring a shuffle - join, groupBy, sort, repartition - as opposed to a narrow one like map or filter that needs no data movement.